

Datacenter TCP (DCTCP)

Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye,
Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, Murari Sridharan

Contents

01 Introduction

02 Communications in Datacenter

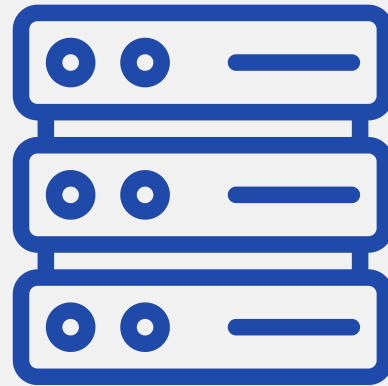
03 DCTCP Algorithm (~3.2)

01

Introduction

Datacenter Newtork

Datacenter



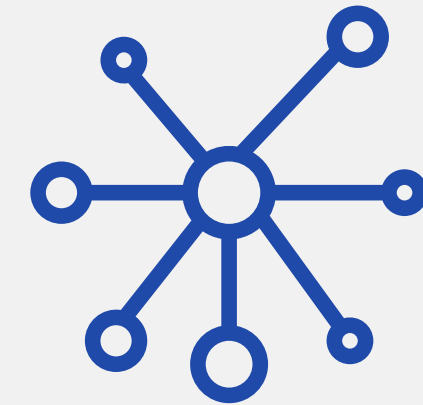
highly available
highly performant computing
and storage infrastructure

using low cost,
commodity components

corresponding trend has emerge



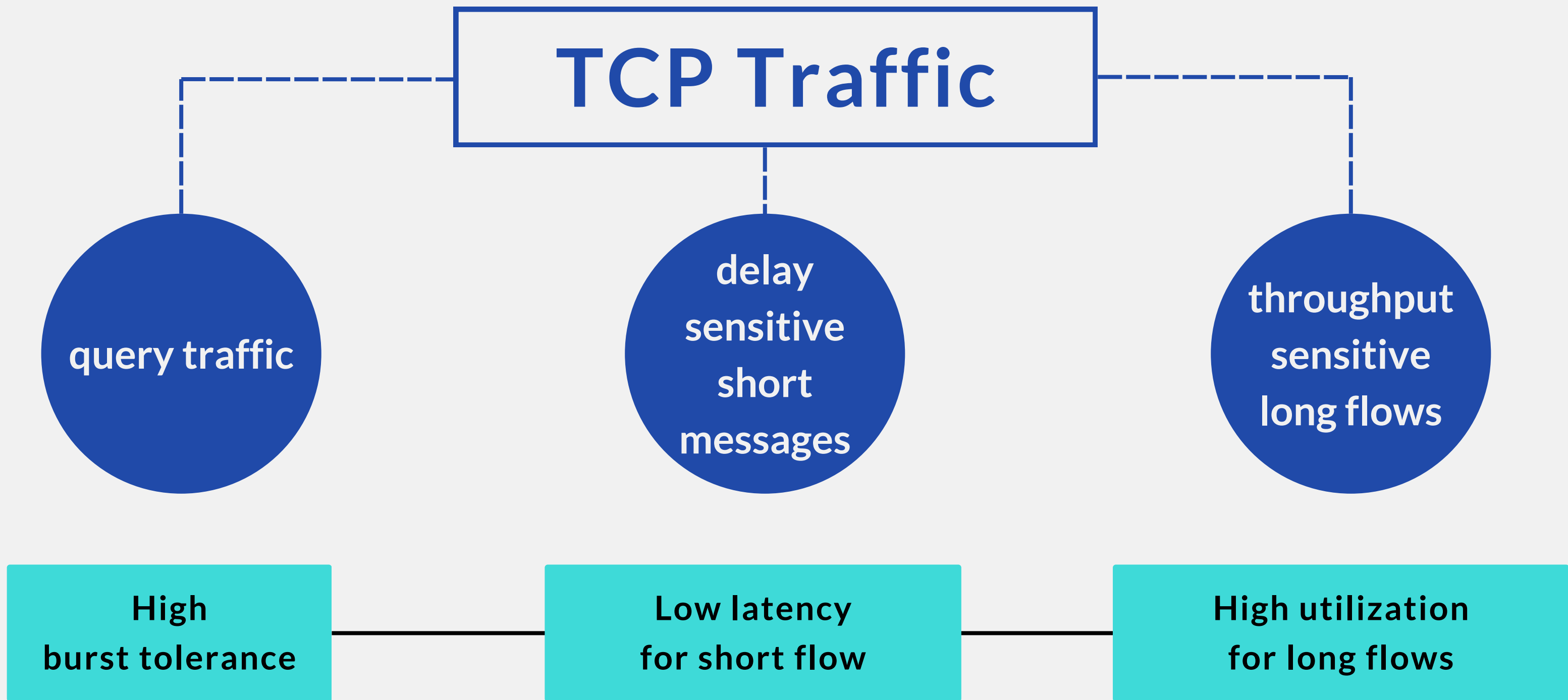
Datacenter Network



economical
easy to manage data centers

using novel architectures
built atop commodity switches

How well
the commodity switches
handle the traffic
of real data center
applications



to meet the **Requirements** of diverse mix of short and long flows
switch **buffer occupancies** need to be **persistently low**
while **maintaining high throughput** for the long flow

DCTCP is designed to do this

DCTCP Design

use ECN

implementable with mechanisms
in existing hardware

congestion control protocols

1. Delay-based protocols
2. Active Queue Management

no claims about suitability of
DCTCP for wide area networks

- $RTT < 250\mu s$ (in absence of queuing)
- need high bandwidths and low latencies
- little statistical multiplexing
- largely homogeneous,
under a single administrative control

02

Communications in Datacenter

Partition/Aggregate

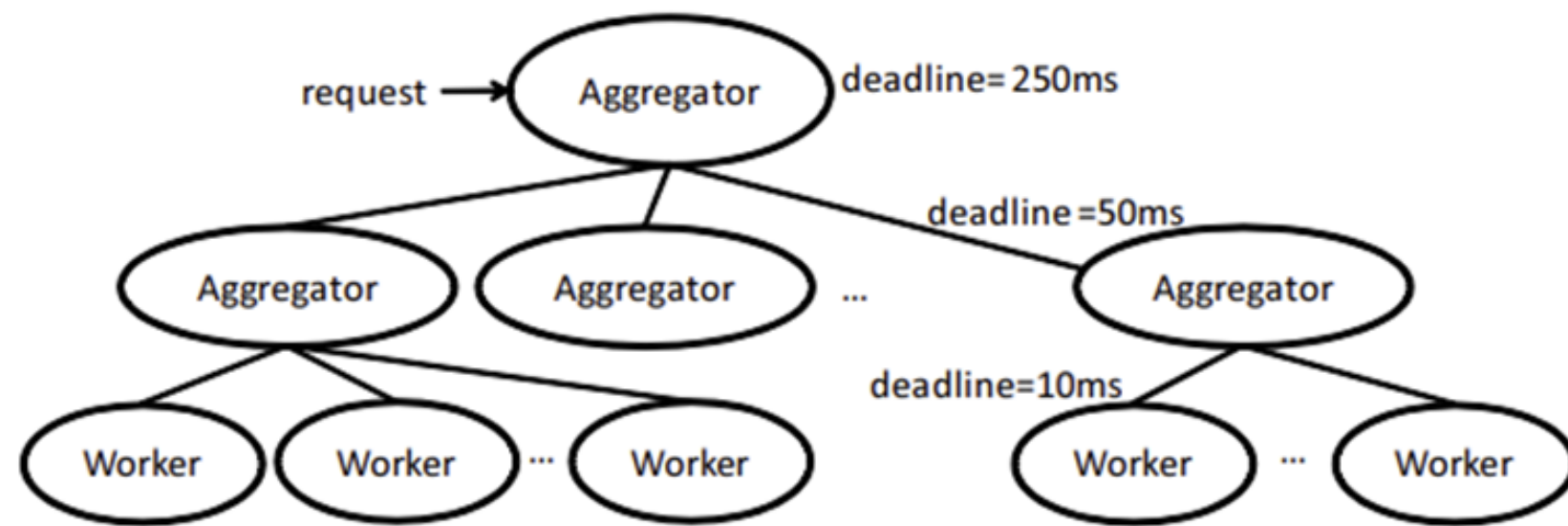


Figure 2: The partition/aggregate design pattern

- Partition request
- Aggregate response

All-up SLA

subtracting Internet and rendering delays, the “backend” part of the application is typically allocated between 230-300ms.

Threaten all-up SLAs for queries

a multi-layer partition/aggregate pattern and iteratively invoking the pattern

Worker nodes are assigned tight deadlines (10-100ms)

to prevent the all-up SLA from being violated

Difficult to meet deadlines using state-of-the-art TCP

resort to complex, ad-hoc solutions

Workload Characterization

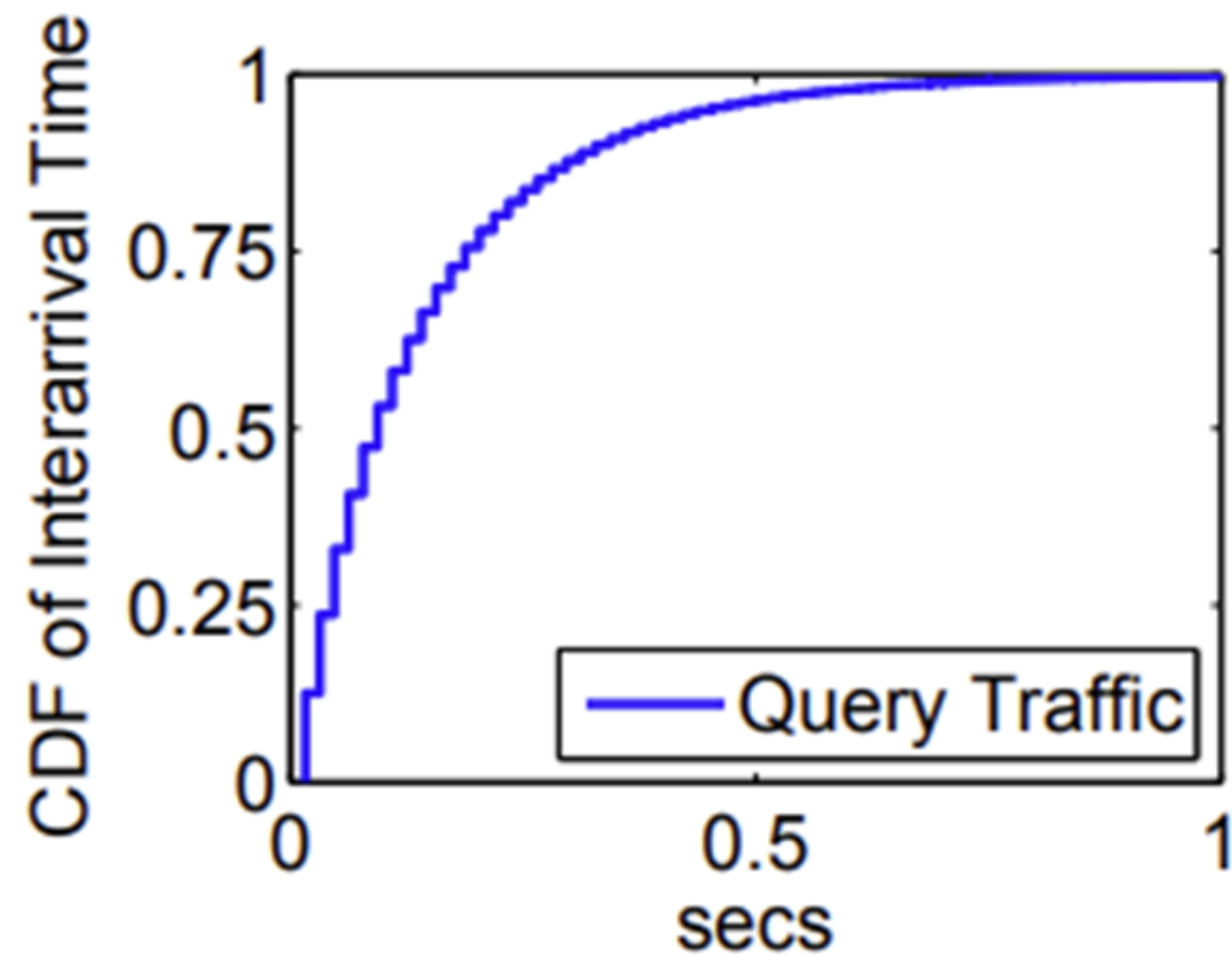
Instrumented a total of over 6000 servers in over 150 racks

Each rack in the clusters holds 44 servers

Each server connects to a Top of Rack switch (ToR) via 1Gbps Ethernet

The ToRs are shallow buffered, shared-memory switches

Query Traffic



Follows the Partition/Aggregate pattern

**Consists of very short,
latency-critical flows**

Size is extremely regular

queries from MLAs to workers being 1.6KB

Responses from workers to MLAs being 1.6 to 2KB.

Background Traffic

Consisting of both large and small flows

Update flow

copy fresh data to the workers (1MB to 50MB)

Time-sensitive short message flows

update control state on the workers (50KB to 1MB)

Inter-arrival time between background flows

- (1) the variance is very high, with a very heavy tail
- (2) embedded spikes occur
- (3) relatively large numbers of outgoing flows occur periodically

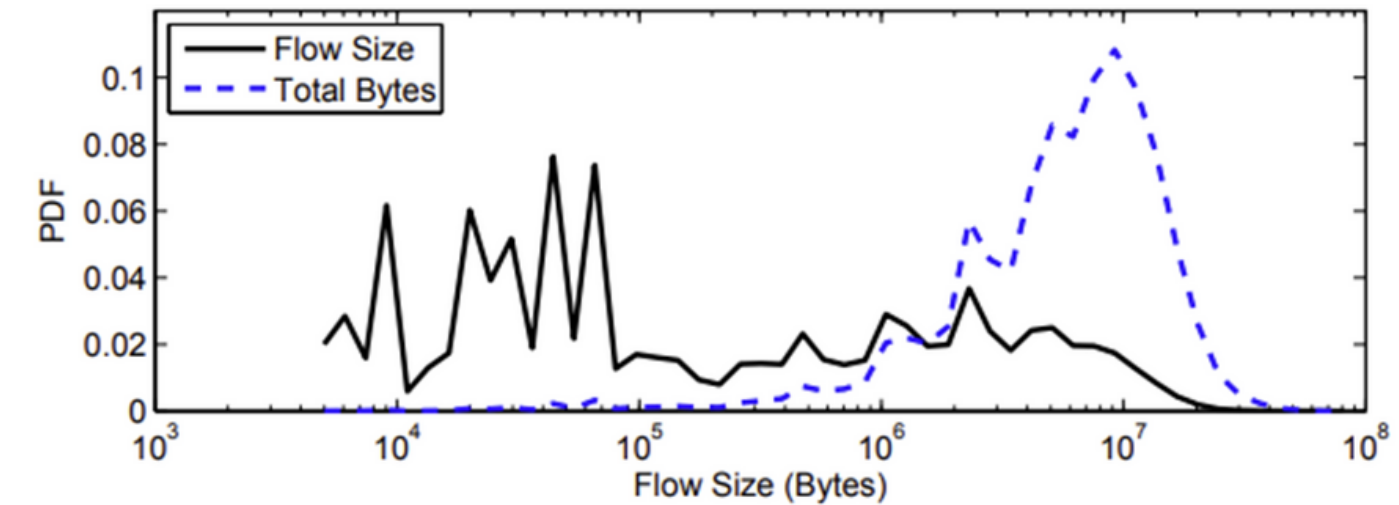
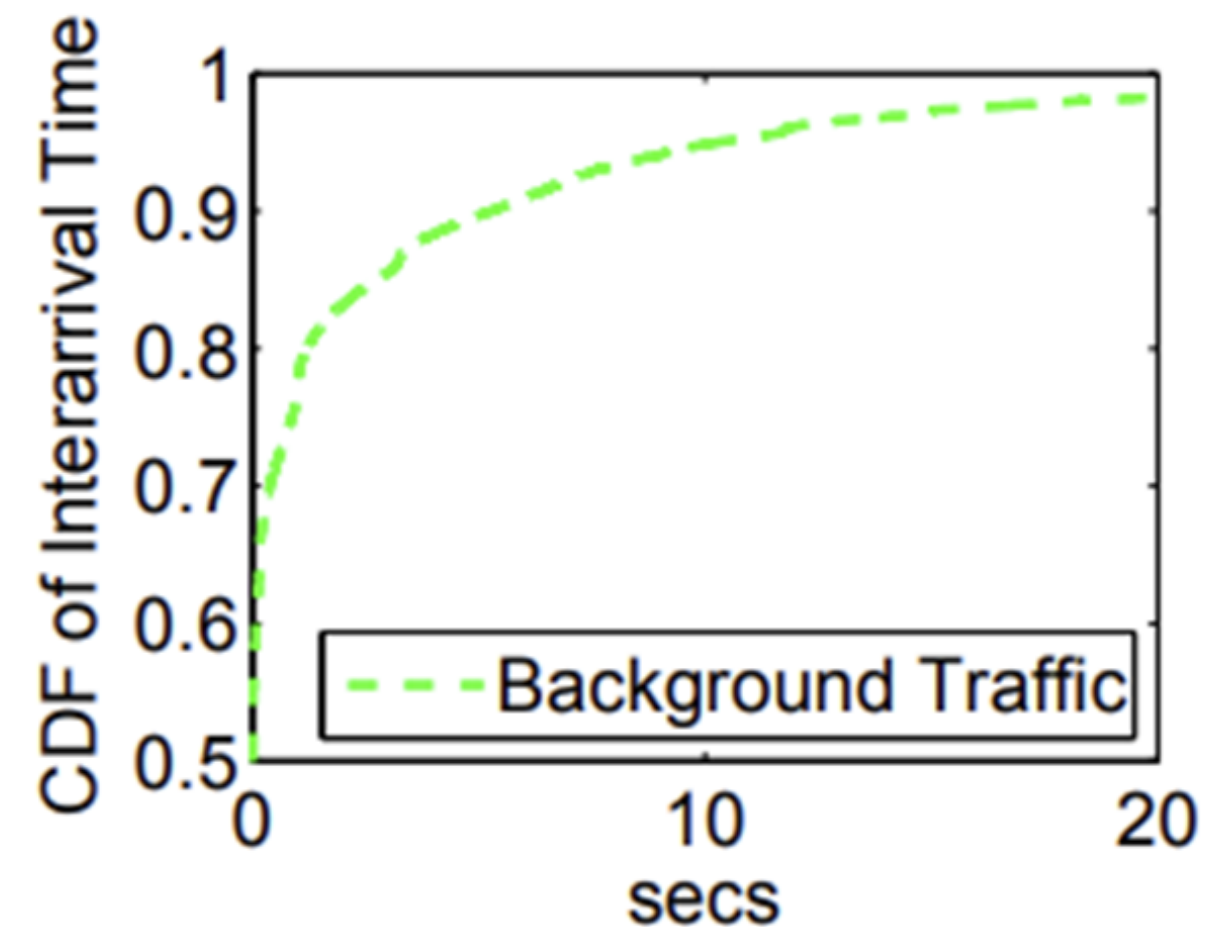


Figure 4: PDF of flow size distribution for background traffic. PDF of Total Bytes shows probability a randomly selected byte would come from a flow of given size.



Flow Concurrency and Size

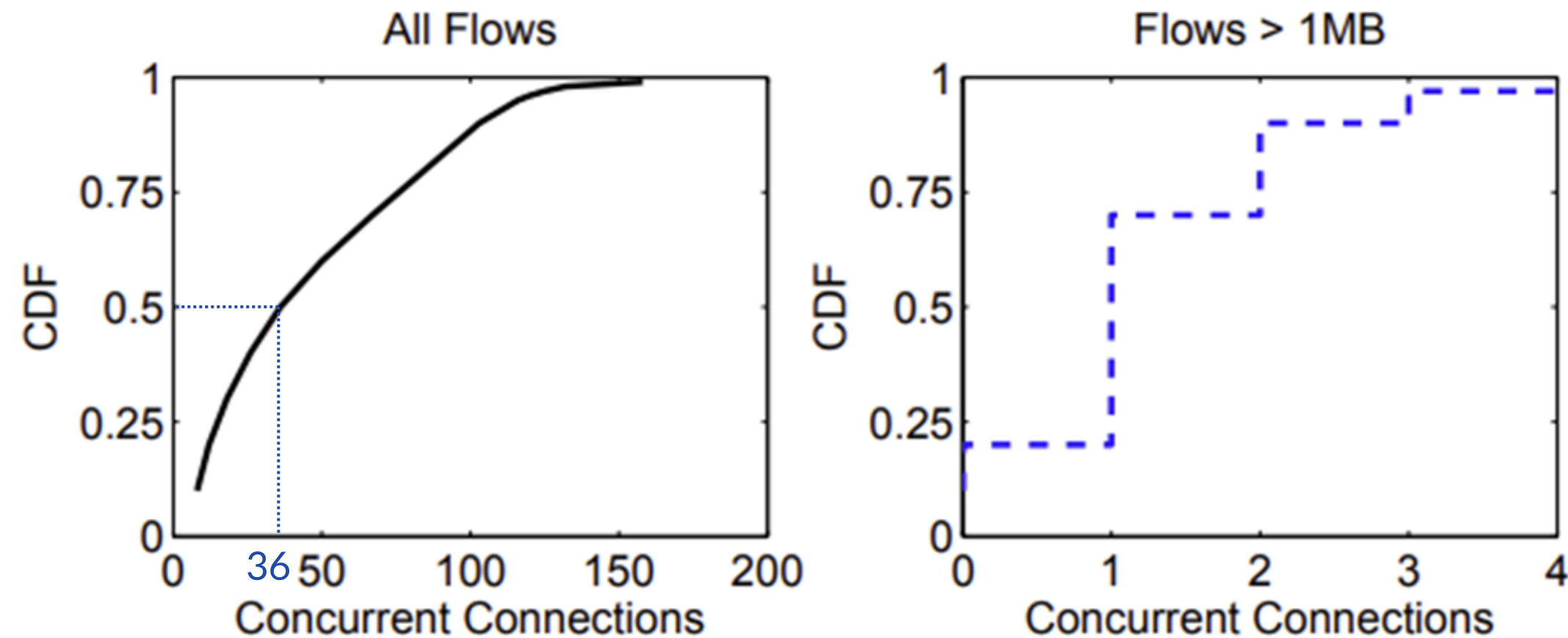


Figure 5: Distribution of number of concurrent connections.

throughput-sensitive large flows, delay sensitive short flows and bursty query traffic
co-exist in a data center network

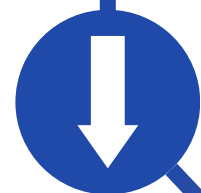
SWTICH



Shared memory



Large multi-ported memories is expensive



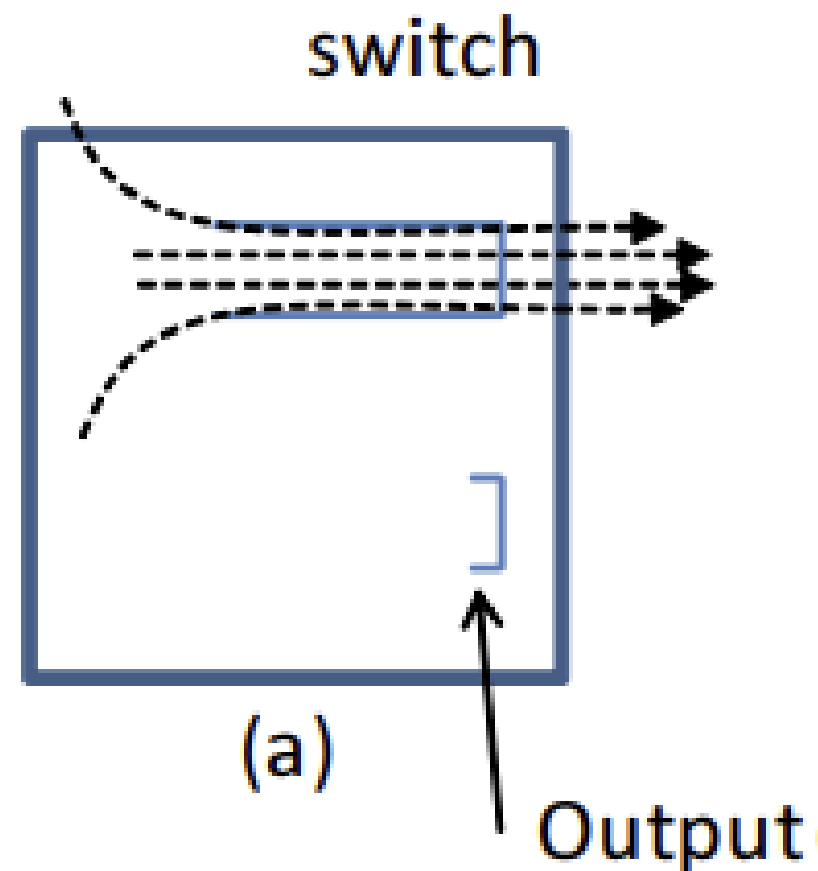
Most cheap switches are shallow buffered



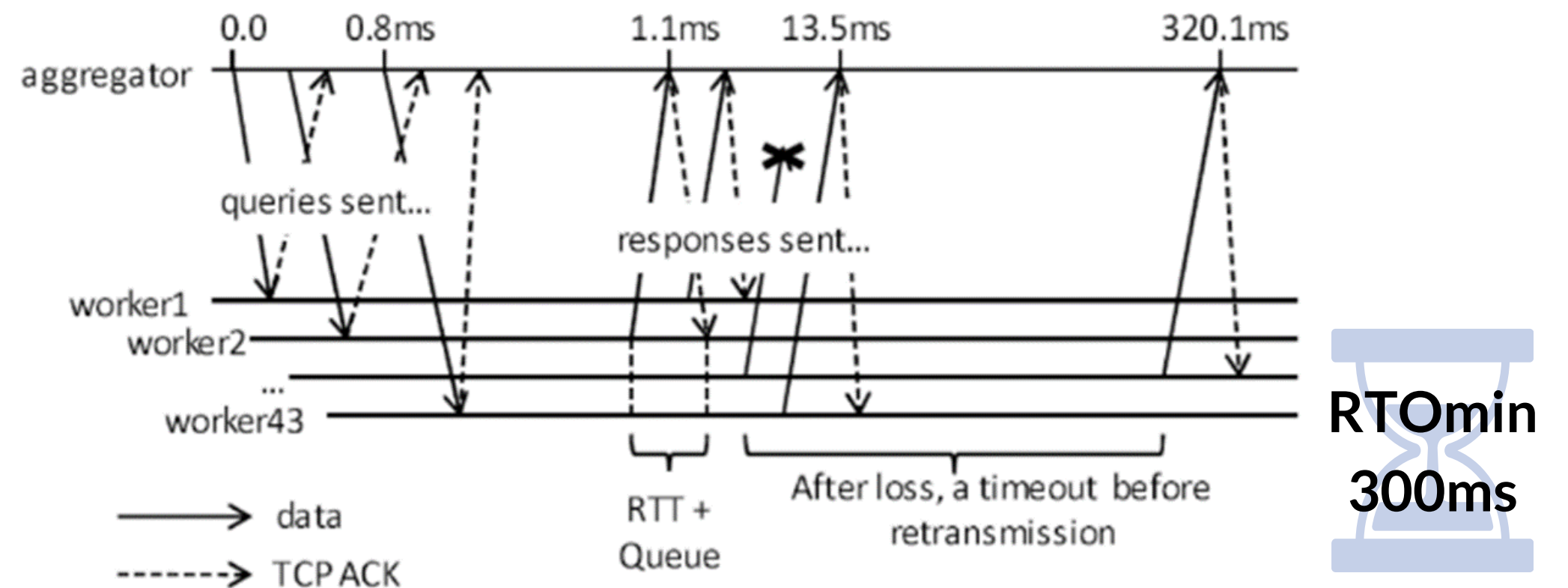
Cause Impairments

Impairments

Incast



many flows converge on same interface
of a switch over a short period of time



arises from use of the Partition/Aggregate pattern

Loss > timeout > degrading quality of result

to mitigate
Incast

Jittering

reduces the response time
at higher percentiles
(by avoiding timeouts)

at the cost of increasing
the median response time
(due to added delay)

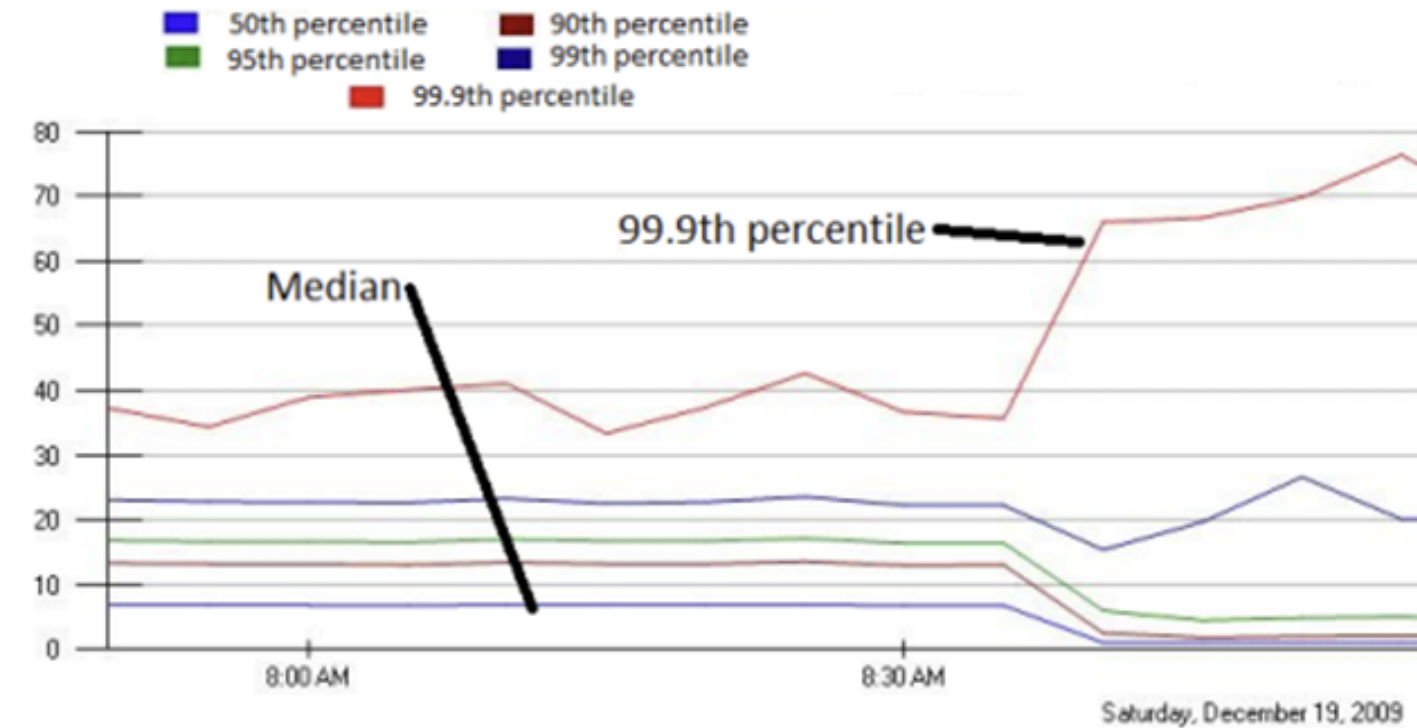
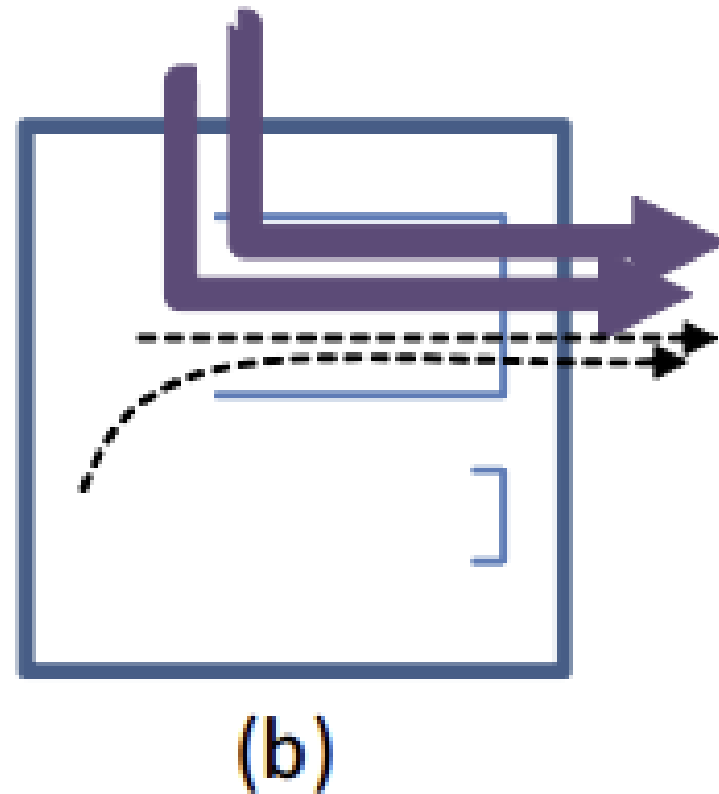


Figure 8: Response time percentiles for a production application having the incast traffic pattern. Forwarded requests were jittered (deliberately delayed) over a 10ms window until 8:30am, when jittering was switched off. The 95th and lower percentiles drop 10x, while the 99.9th percentile doubles.

Impairments

Queue buildup



long and short flows
traverse the same queue

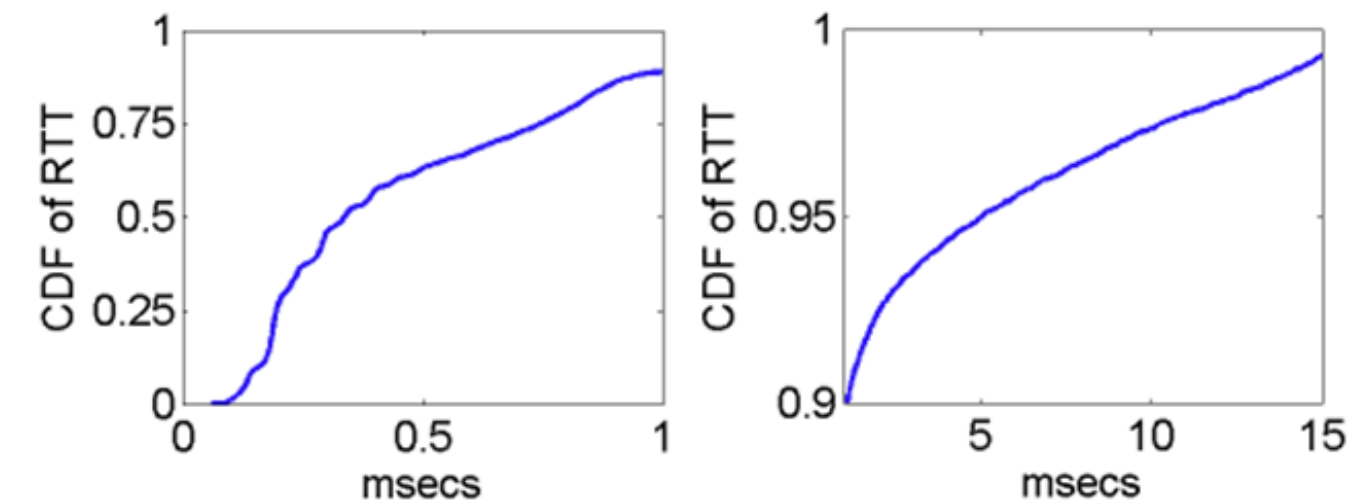
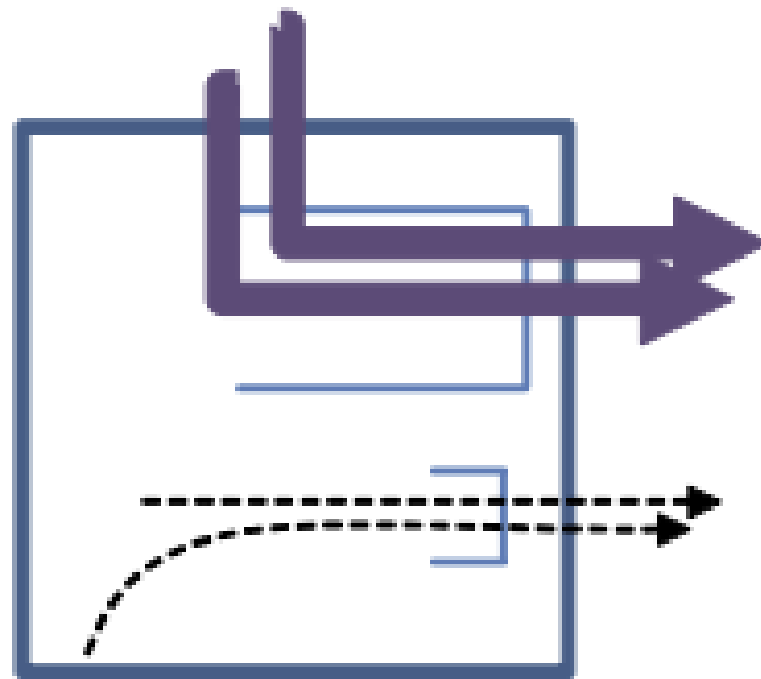


Figure 9: CDF of RTT to the aggregator. 10% of responses see an unacceptable queuing delay of 1 to 14ms caused by long flows sharing the queue.

even when no packets are lost,
the short flows experience increased latency
as they are behind packets from the large flows

Impairments

Buffered Pressure



(c)

reduces the buffer space
to absorb bursts

buffer space is a shared resource

long, greedy TCP flows build up queues

packet loss and timeouts
without synchronized flows

03

DCTCP Algorithm

Goals of DCTCP

With shallowed switch

01.

High burst tolerance

02.

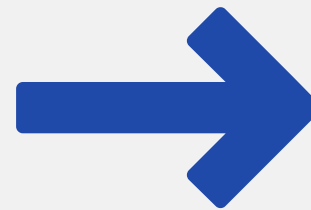
High throughput

03.

Low latency

DCTCP design

- small queue occupancies
- without loss of throughput



Use ECN

Main Idea

ECN Marks	TCP	DCTCP
1 0 1 1 1 1 0 1 1 1	Cut window by 50%	Cut window by 40%
0 0 0 0 0 0 0 0 0 1	Cut window by 50%	Cut window by 5%

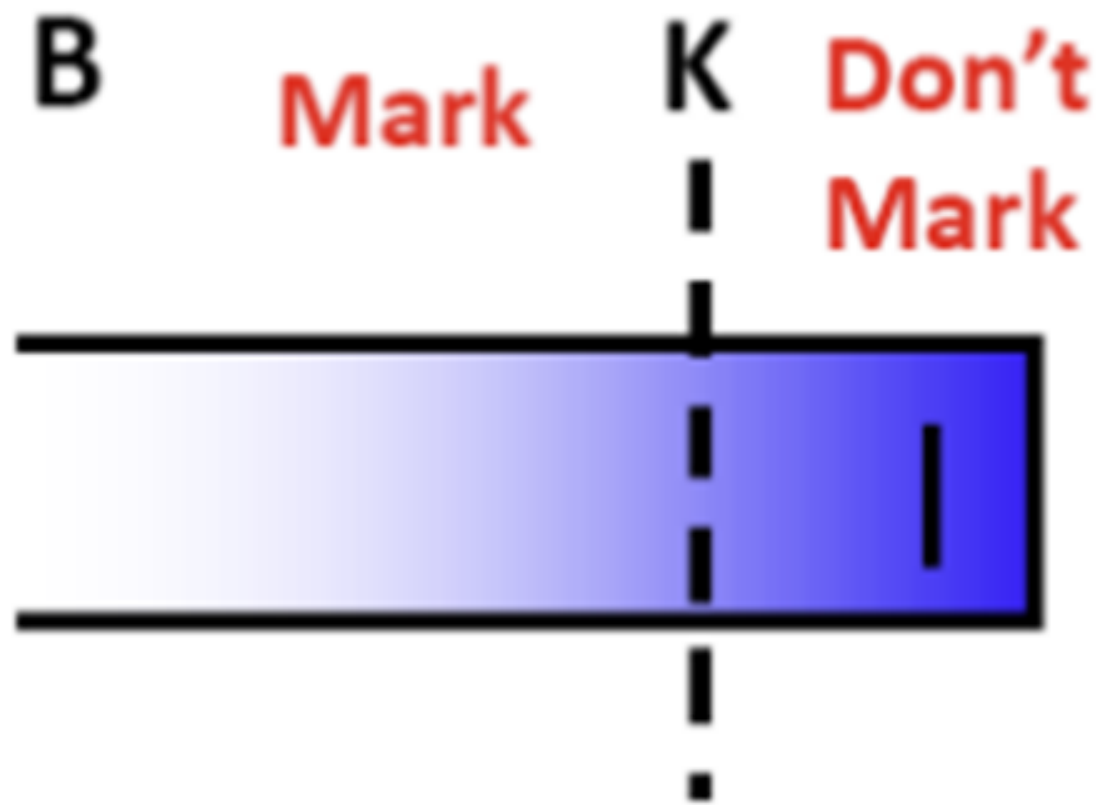
deriving multi-bit feedback from the information present in the single-bit sequence of marks

Presence of Congestion (TCP) VS **Extent** of Congestion (DCTCP)

Reduce Window size based on **Fraction** of marked packet

ALGORITHM

Simple Marking at the Switch



mark CE codepoint
when **queue occupancy** $> K$

ensures that sources are **quickly**
notified of the queue overshoot

ALGORITHM

ECN-Echo at the Receiver

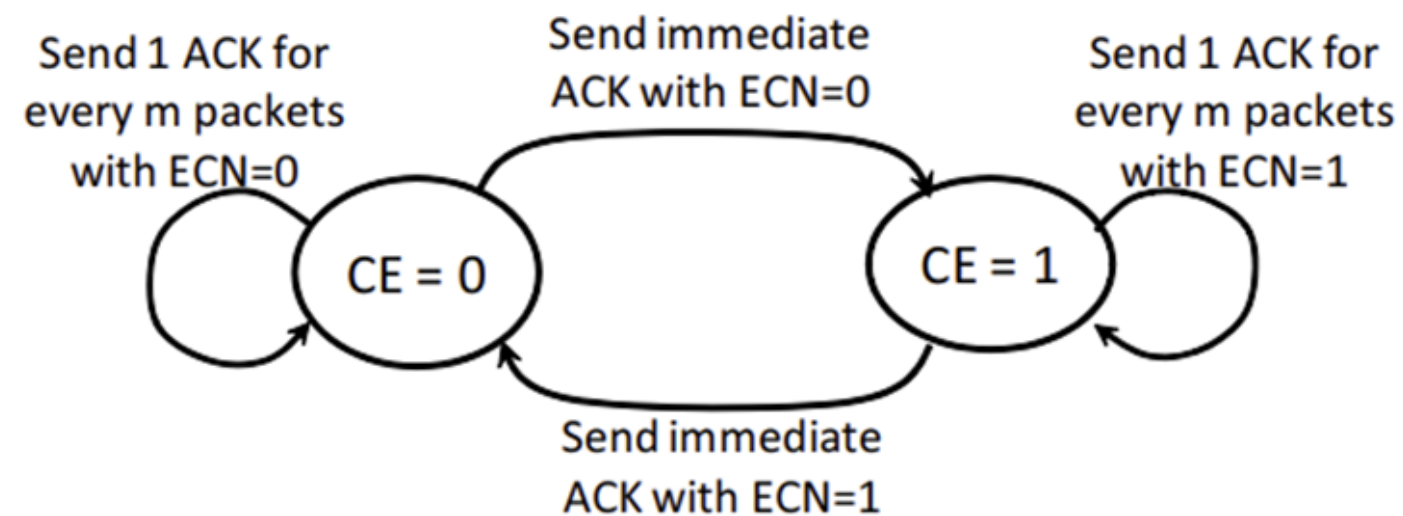
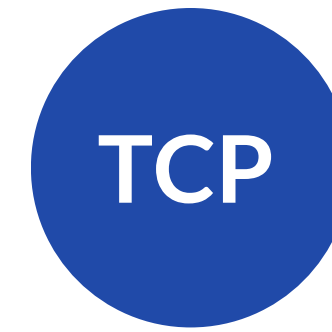


Figure 10: Two state ACK generation state machine.

Use delayed Ack in DCTCP



Set ECN-Echo Until..

receives confirmation from the sender
(through the CWR flag) that the congestion
notification has been received



Accurately convey marked packets

ACK every packet, setting ECN-Echo flag
if and only if packet has a marked CE codepoint

ALGORITHM

Controller at the Sender

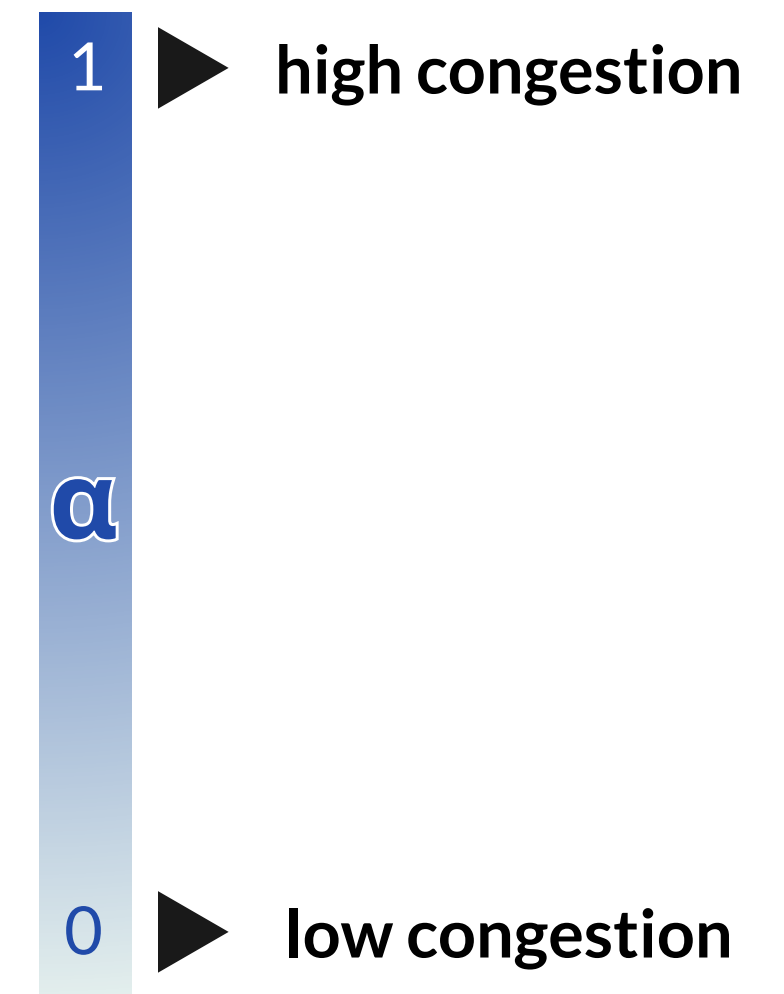
maintains an estimate of the fraction of packets that are marked, called α

updated once for every window of data (roughly one RTT) as follows :

$$\alpha \leftarrow (1 - g) \times \alpha + g \times F$$

F fraction of packets that were marked in the last window of data

$0 < g < 1$
weight given to new samples against the past in the estimation of α



ALGORITHM

Controller at the Sender



Always Cut window size in half

large mismatch
between input rate to the link and available capacity

leads to buffer underflows and loss of throughput



$\text{cwnd} \leftarrow \text{cwnd} \times (1 - \alpha/2)$

maintains low queue length,
while still ensuring high throughput

Benefit

Queue Buildup

- DCTCP start reacting as soon as queue length exceeds K

>> Reduce queueing delay and more buffer space to absorb transient micro-bursts

Incast

- DCTCP starts marking early
- receive enough marks during the first 1 or 2 RTTs to tame the size of follow up bursts

>> prevents buffer overflows and resulting timeouts

Buffered pressure

- Queue length does not grow exceedingly large

>> Not exhaust the buffer resources

Q&A